

M03 / LEARNER BOOK

SQL for engineering

Standalone Zero-to-Job-Ready Data Engineer 2026

16 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module

DE03-01 | INNER JOIN and LEFT JOIN

DE03-02 | Cardinality, row multiplication and join validation

DE03-03 | UNION and UNION ALL

DE03-04 | String functions for cleaning and standardization

DE03-05 | Dates, timestamps and time analysis

DE03-06 | Subqueries and nested logic

DE03-07 | CTEs and readable multi-step SQL

DE03-08 | Window functions and PARTITION BY

DE03-09 | ROW_NUMBER, ranking and LAG/LEAD

DE03-10 | Advanced aggregation and multi-grain reasoning

DE03-11 | PostgreSQL setup, schemas and dialect awareness

DE03-12 | Constraints and transactions

DE03-13 | Indexes and EXPLAIN basics

DE03-14 | Views and reusable database logic

DE03-15 | Integrated unseen SQL challenge

DE03-16 | Timed SQL interview set and explanation

3

4

7

10

13

16

19

22

25

28

31

34

37

40

43

46

49

START HERE

How to use this module

Open the matching practice folder from the course root: 01_M00_M03/labs. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is written for a beginner. Work through the lessons in order. Read the new words, watch the assigned instructor range, perform the printed example on your laptop, then save an independent attempt before opening the explained review. Ready-made revision is included; you do not need to rewrite the book as notes.

This cumulative package contains M00-M03, G01 and G02. It keeps the completed earlier modules alongside the new SQL modules. The full standalone course continues to M19; the later modules are not delivered as completed books in this batch. There is no fixed time-to-job promise.

Instructor videos open online and use their creators' data. Exact start and stop times are printed with each assigned range. Stop manually at the end: a YouTube start link does not enforce the stop. Published chapter titles, boundaries and runtime were inspected, not complete playback. Read the purpose and any software difference beside the link. A lesson can include written details that its supporting video does not cover; coverage limits remain explicit in WATCH_GUIDE.html.

Study without a Mentor app for now. Keep your own attempts and result evidence using the supplied template. The books contain learner tasks and changed retries; the separate review books contain their answers. The cumulative ZIP includes all required fictional inputs. Never open an answer script before saving your genuine attempt.

DE03-01

INNER JOIN and LEFT JOIN

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

03:29:30-03:40:53

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-14. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[03:29:30-03:40:53. Watch](#) inner and left joins. Our own fixtures include unmatched customers and an empty order. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

M02 kept one item per order in a simplified table. Real systems often store customers, orders and order items separately. You will attach a customer name to each order without losing track of unmatched records. Complete M02, keep the loaded course schema, and use course.orders and course.customers for this lesson. They are new training tables; order IDs begin with O rather than the M02 numeric-looking IDs.

A join combines records according to a matching condition. A key identifies a record or links it to another table. Here customers has one row per customer, identified by customer_id. orders has one row per order, identified by order_id, and may refer to a customer_id. These fields have different roles: several orders can belong to one customer.

Picture it before writing it

Asha's customer card is C01. Orders O101 and O103 both refer to C01, so both may legitimately display Asha after a join. The two displayed names do not mean the customer table contains duplicate Asha cards. O106 has no customer_id; we have an order but no customer identity for it. Tara, C05, has no order in the training source.

Question Appropriate starting point Expected grain

Orders with a matched customer

orders joined to customers One row per matched order

Every order, named if possible

orders LEFT JOIN customers One row per order here

Every customer, with any orders

customers LEFT JOIN orders One row per customer/order match, plus unmatched customer

The last result is not necessarily one row per customer because Asha has two orders. Predicting that multiplicity is the next lesson's main work.

Follow with me once

```
SELECT o.order_id, c.customer_name, o.shipping_fee
FROM course.orders AS o
INNER JOIN course.customers AS c
  ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

AS o and AS c give the tables short names within this query. o.order_id means the order_id belonging to o. ON states the matching relationship. INNER JOIN keeps matched pairs. The output is O101 Asha/20, O102 Ravi/10, O103 Asha/0, O104 Mina/15 and O105 Iqbal/5. Five rows appear; O106 is excluded because its null customer_id does not match a known customer key.

Now preserve every order:

```
SELECT o.order_id, o.customer_id, c.customer_name
FROM course.orders AS o
LEFT JOIN course.customers AS c
  ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

Six rows appear. O106 has a NULL name as well as its NULL customer_id. LEFT JOIN preserves each left-side row, with right-side nulls if there is no match. It does not promise exactly one output per left row when several matches exist. Here customer_id is unique in customers, so each order matches at most one customer.

ON versus WHERE can change preservation

Suppose you want every order and an East-region customer name where one matches. Put the right-side restriction inside the matching rule:

```
SELECT o.order_id, c.customer_name
FROM course.orders AS o
LEFT JOIN course.customers AS c
  ON o.customer_id = c.customer_id
  AND c.region = 'East'
ORDER BY o.order_id;
```

All six orders remain, but only O101, O102 and O103 get names. If instead you put WHERE c.region = 'East' after the join, the null-extended rows fail that filter, and only those three orders remain. Neither form is automatically wrong; they answer different questions. State which records must be preserved before choosing where to put the condition.

If you are stuck

“Column customer_id is ambiguous” means the name exists on both sides and PostgreSQL needs you to identify which one. Prefix it with o. or c. An unexpectedly large result can mean the ON condition is missing or matches non-unique values. Do not join customer names merely because they look readable; names can repeat or change. Use the declared relationship key.

Try it yourself

Start with all customers and attach their order IDs using LEFT JOIN. Display customer_id, customer_name and order_id, sorted by customer_id and order_id. Predict how many rows Asha produces and what Tara's order field shows.

Changed retry - after your first review

Return every order but attach a customer name only when the customer belongs to North. Keep all original orders, including those outside North and O106. Explain why WHERE region = 'North' would fail that preservation rule.

Revision in your own words

INNER JOIN keeps matching pairs. LEFT JOIN also preserves unmatched left rows. Use the relationship key, qualify ambiguous names and predict multiplicity. ON defines matches; WHERE can remove rows after the join. Record both row counts and unmatched cases, not only a familiar-looking name.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-01 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-02

Cardinality, row multiplication and join validation

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

51:10-01:08:04

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-01. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 51:10-01:08:04.](#) [Optional joins](#) refresher after stating both input grains. Our book explicitly demonstrates row multiplication and proves the corrected totals; the video is not a replacement for that validation. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A silent join error can multiply an order's shipping fee without producing any SQL error. You will identify each table's grain, predict a one-to-many join, measure the multiplication, and choose a repair that respects what each amount represents. Complete DE03-01.

Cardinality describes how many matches a relationship permits. Fan-out is the multiplication of a left record across several matching right records. An invariant is a fact expected to remain true across a transformation, such as the total fee attached once to each original order. A composite key identifies a record using more than one field together.

course.order_items has one row per order line. Its key is (order_id, line_no). O101 has two lines: two notebooks amounting to 99, and three pens amounting to 30. Its order-level shipping fee is 20. The correct combined amount is $99 + 30 + 20 = 149$. The fee belongs to the whole order, not independently to each line.

Predict the shape first

There are six orders, eight item lines and five distinct order IDs represented in items. O106 has no item lines. An INNER JOIN from orders to items therefore produces eight matched line rows and drops O106. A LEFT JOIN produces those eight line rows plus one null-extended O106 row: nine results. The left join retains every order but does not preserve the original six-row grain.

Follow with me once

```
SELECT o.order_id, i.line_no,
       i.quantity * i.unit_price AS item_amount,
       o.shipping_fee
FROM course.orders AS o
JOIN course.order_items AS i
  ON o.order_id = i.order_id
ORDER BY o.order_id, i.line_no;
```

Look at O101's two rows: each displays shipping_fee 20. That repetition is expected at line grain. It becomes a financial error only if you add the fee as though each displayed copy were a separate charge. A joined field can be safe to display and unsafe to sum at the new grain.

```
SELECT COUNT(*) AS joined_rows,
       COUNT(DISTINCT o.order_id) AS represented_orders,
       SUM(o.shipping_fee) AS repeated_fees
FROM course.orders AS o
JOIN course.order_items AS i
  ON o.order_id = i.order_id;
```

```
SELECT COUNT(*) AS source_orders,
       SUM(shipping_fee) AS source_fees
FROM course.orders;
```

The joined summary is 8 rows, 5 represented orders and 75 repeated fees. The original is 6 orders and 50 fees. O101 adds one extra 20 and O105 adds one extra 5, so the sum is inflated by 25. O106's omitted fee happens to be zero; the fee-total comparison alone would not reveal its lost order. That is why row coverage and amount reconciliation are separate checks.

Repair the grain, not the appearance

For one order, aggregate item values first and attach its fee once:

```
SELECT o.order_id, o.shipping_fee,
       (SELECT SUM(i.quantity * i.unit_price)
        FROM course.order_items AS i
        WHERE i.order_id = o.order_id) AS item_total
FROM course.orders AS o
WHERE o.order_id = 'O101';
```

The result has one O101 row, fee 20 and item_total 129. Its payable amount is 149. This is a correlated subquery: the inner query uses the outer order ID. DE03-06 explains that mechanism; DE03-07 builds a more reusable all-order version with a CTE. Here the important repair is producing item totals at order grain before combining order-level measures.

SUM(DISTINCT shipping_fee) is not that repair. It removes repeated equal fee values across different orders too. If two genuine orders each paid 10, a distinct sum would count 10 once instead of 20. An amount is not an order key.

Other join shapes you should recognise

A RIGHT JOIN preserves the right side; reversing table positions and using LEFT JOIN often expresses the same intent more readably. FULL OUTER JOIN preserves unmatched records from both sides. Those can help reconcile independent deliveries, but matching duplicates still multiply pairs. A CROSS JOIN pairs every left row with every right row: six orders and five customers make thirty pairs. Use that only when every combination is the intended result, not as a substitute for a missing ON condition.

Try it yourself

Inspect O105's item lines. Calculate its item total and correct fee-inclusive amount by hand, then use a query to verify the item total. Calculate the wrong fee-inclusive amount produced by adding its fee once per item line. State the grain of that wrong intermediate result.

Changed retry - after your first review

Count all rows from a LEFT JOIN of orders to items. Also count non-null item line numbers and distinct order IDs. Explain why these three counts differ, and identify the row causing the difference between the first two counts.

Revision in your own words

Declare both input grains, matching keys, expected cardinality and final grain. Count source rows, joined rows, represented keys and unmatched cases. Aggregate line measures to order grain before adding order-level charges. DISTINCT on a money value cannot replace a key-aware repair.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-02 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-03

UNION and UNION ALL

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:08:04-01:15:10

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-02. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:08:04-01:15:10.](#)

[Watch](#) UNION. Our lesson explains UNION ALL, duplicate meaning and introduces INTERSECT/EXCEPT. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

Joins attach related fields side by side. Sometimes you need to append compatible records from two deliveries instead. You will use UNION ALL to preserve occurrences and UNION when the question asks for unique selected rows. Complete DE03-02.

A set operation combines query results vertically. Each branch must return the same number of columns in the corresponding order, with compatible types. Output labels come from the first branch. Picture putting two receipt stacks on top of each other. That operation is different from attaching a customer's name to a receipt.

Follow with me once

```
SELECT order_id, category
FROM course.sales
WHERE ordered_on = DATE '2026-09-01'
UNION ALL
SELECT order_id, category
FROM course.sales
WHERE ordered_on = DATE '2026-09-02'
ORDER BY order_id;
```

Four rows remain: 0007 Paper, 0008 Writing, 0009 Paper and 0010 Writing. UNION ALL retains every occurrence from both branches. The final ORDER BY applies to the combined result. Neither branch needs a separate ORDER BY for this question.

Now select only category and use UNION:

```
SELECT category FROM course.sales
WHERE ordered_on = DATE '2026-09-01'
UNION
SELECT category FROM course.sales
WHERE ordered_on = DATE '2026-09-02'
ORDER BY category;
```

Only Paper and Writing remain because UNION removes duplicate whole output rows. If UNION ALL were used with those category-only branches, Paper would appear twice and Writing twice. Neither version deletes anything from course.sales. They request different result shapes.

A duplicate occurrence can be legitimate

Two sales of Pen are two sales, not automatically one duplicated event. If a daily ingestion replays the same order_id by mistake, you need a business key and a chosen replay policy to repair it. UNION on a truncated list of fields may collapse legitimate events, while including a delivery timestamp could keep replayed events distinct. Think about identity before using an operation that discards rows.

You can add a literal source label to retain provenance:

```
SELECT 'day_1' AS source_batch, order_id
FROM course.sales
WHERE ordered_on = DATE '2026-09-01'
UNION ALL
SELECT 'day_2' AS source_batch, order_id
FROM course.sales
WHERE ordered_on = DATE '2026-09-02'
ORDER BY source_batch, order_id;
```

The label lets you trace which branch supplied a result. It is an original query value, not a field that already existed in the table. Because it becomes part of the selected row, it also affects uniqueness if you later change UNION ALL to UNION.

Related operations

INTERSECT returns selected rows common to both results. EXCEPT returns selected rows from the first result absent from the second. Their default forms remove duplicate rows; ALL variants have multiplicity rules that are outside this first lesson. For now, use them on small key lists you can inspect. They compare output rows, not a hidden matching condition like a JOIN's ON clause.

If you are stuck

An unequal-column error means the branches disagree about their output shape. Compare both field lists in order. Compatible types are not enough for compatible meanings: a numeric unit price placed under a quantity heading would still be wrong. Do not fix a type problem by turning every value into text without deciding what that shared output column represents.

Try it yourself

Append the category values from course.practice_sales on October 1 and October 2 using UNION ALL. Repeat with UNION. Predict both row counts and explain which repeated category is removed in the second result.

Changed retry - after your first review

Find categories appearing in course.retry_sales on both November 1 and November 2 using INTERSECT. Then find categories from November 1 absent on November 2 using EXCEPT.

Revision in your own words

JOIN combines matching records side by side; UNION operations append compatible result rows. UNION ALL preserves occurrences, UNION removes duplicates of the whole selected row. Align column meaning and type. Preserve source labels when provenance matters, and distinguish legitimate repeated values from replayed business events.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-03 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-04

String functions for cleaning and standardization

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:15:10-01:26:39

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-03. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:15:10-01:26:39.](#) [Watch](#) string functions; Alex uses MySQL. PostgreSQL uses LENGTH, TRIM and SUBSTRING forms shown in our book; there is no native MySQL-style LOCATE or IFNULL to copy. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

In M02 you trimmed ordinary surrounding spaces and changed case. You will now treat blank cleaned labels as missing, inspect the transformation and avoid inventing a product identity from similar-looking text. Complete DE03-03. Cleaning is a controlled rule, not a command to make every string look alike.

Standardisation converts known variants into a consistent representation. Normalisation can mean several things in computing; in this lesson it means a text-formatting rule, not relational database normalisation. Provenance means knowing where a value came from and how it changed. A mapping is an explicit association between a source label and a known target value.

Follow with me once

```
SELECT source_id, item_raw,  
       LENGTH(item_raw) AS raw_length,  
       NULLIF(LOWER(TRIM(item_raw)), '') AS item_clean  
FROM course.raw_labels  
ORDER BY source_id;
```

A's raw length is 6 and its clean value is pen. B's length is 3 and its clean value is pen. C's length is 0 and its clean value is NULL. D has null length and null clean value. E has length 11 and cleans to note book. F has length 9 and keeps pen blue with two internal spaces. Inspect these alongside the raw values rather than trusting a visual glance at blank-looking cells.

Read the nested expression from the inside out: TRIM removes surrounding spaces; LOWER changes case; NULLIF compares that cleaned result with the empty string and returns NULL when they are equal. If the raw input is already NULL, it remains missing through this expression. The empty string is written as two single quotes with nothing between them.

Validate the effect of the rule

```
SELECT COUNT(*) AS raw_rows,  
       COUNT(item_raw) AS raw_non_null,  
       COUNT(NULLIF(LOWER(TRIM(item_raw)), ''))  
         AS usable_clean_labels  
FROM course.raw_labels;
```

There are six raw rows, five non-null raw labels and four usable non-blank cleaned labels. One empty string became explicitly missing; the original NULL stayed missing. Cleaning did not reduce the source row count. If you later filter to only usable labels, that deliberate rejection step should be counted and explained separately.

A pattern match is not proof of identity

```
SELECT source_id, item_raw  
FROM course.raw_labels  
WHERE LOWER(TRIM(item_raw)) LIKE 'pen%'  
ORDER BY source_id;
```

A, B and F match. The % wildcard means zero or more following characters. It catches pen and pen blue, which may represent different products. LIKE is useful for finding candidates to inspect; it does not prove those candidates share one product key. An underscore wildcard means one character, so literal wildcard characters require deliberate escaping when that situation arises.

Removing every internal space would turn note book into notebook, but it would also change other labels indiscriminately. Use a reviewed lookup mapping if you know that a particular spelling refers to a particular product. Preserve an unmapped category when you do not know. The same caution applies to replacing punctuation, stripping leading zeros from IDs or merging differently cased identifiers whose system treats case as meaningful.

If you are stuck

Check LENGTH and the raw value to distinguish empty from null. TRIM's ordinary-space rule may not remove tabs or non-breaking spaces; do not claim universal whitespace repair. If two raw labels now share one clean value, inspect whether that collision is expected. A standardised display label is not necessarily a unique identifier.

Try it yourself

Return only source IDs whose cleaned label under the guided rule is missing. Keep the raw label in the result. Then count distinct non-null cleaned labels and explain why that count is lower than the number of usable cleaned rows.

Changed retry - after your first review

Using only the three literal inputs below, calculate raw value, cleaned value and cleaned length: 'Bag ', 'BAG' and ' '. You can use UNION ALL to create three SELECT statements. Decide whether a blank cleaned value should have length zero or a null length under the chosen missing-label rule.

Revision in your own words

Keep raw data and describe each cleaning rule. `NULLIF(TRIM(...), '')` can represent blank labels as missing. Count raw rows, usable values and unique cleaned values separately. Pattern matches identify candidates, not reliable product identity. Do not erase unknown mappings or leading zeros to make data look tidy.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-04 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-05

Dates, timestamps and time analysis

Watch first

[1. SQL for Data Analytics - Learn SQL in 4 Hours](#)

02:10:02-02:20:26

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-04. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: SQL for Data Analytics - Learn SQL in 4 Hours, 02:10:02-02:20:26.](#) [Watch date/time](#) functions. The book supplies the India/UTC boundary example and half-open timestamp filter. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A sale close to midnight can belong to one day in India and a different day in UTC. You will distinguish a calendar date from an instant, select a local business day with exact boundaries and avoid missing the last second of a reporting period. Complete DE03-04.

A date is a calendar day. A timestamp without time zone is a date and clock reading without a uniquely specified global instant. PostgreSQL `timestampz` represents an instant and displays it in the session's time zone; it does not retain the original named zone supplied by the source. UTC is a shared time reference. `AT TIME ZONE` can convert a `timestampz` instant to the local clock reading for a chosen zone.

Picture the boundary

The three events in `course.events` are chosen around midnight in India. Their instants are T1 at August 31 18:30 UTC, T2 at September 1 18:29:59 UTC, and T3 one second later. At Asia/Kolkata, those are September 1 00:00, September 1 23:59:59 and September 2 00:00. The same instants have different calendar labels depending on the zone used for the question.

Follow with me once

```
SELECT event_id,  
       happened_at AT TIME ZONE 'UTC' AS utc_clock,  
       happened_at AT TIME ZONE 'Asia/Kolkata' AS india_clock,  
       (happened_at AT TIME ZONE 'Asia/Kolkata')::date  
       AS india_date  
FROM course.events  
ORDER BY happened_at;
```

The `::date` suffix casts the local clock reading to its calendar date. This is PostgreSQL cast syntax; `CAST(expression AS date)` is another form. The timezone conversion happens before the date extraction, so the date refers to the intended business location rather than your session default.

Filter one full local day

```
SELECT event_id
FROM course.events
WHERE happened_at >= TIMESTAMPTZ '2026-09-01 00:00:00+05:30'
      AND happened_at < TIMESTAMPTZ '2026-09-02 00:00:00+05:30'
ORDER BY happened_at;
```

T1 and T2 qualify; T3 is excluded at the next day's exact boundary. The +05:30 offset is appropriate for these India dates. Other regions may have daylight-saving changes, so do not assume every local calendar day everywhere contains exactly 24 hours or that one fixed offset is valid all year. Naming the reporting zone makes the requirement clear.

Do not use BETWEEN September 1 midnight AND September 2 midnight for this full-day rule: BETWEEN includes both boundaries and would also include T3. Do not use an imagined last instant of 23:59:59, because real timestamps can contain fractional seconds after it. Inclusive start and exclusive next start handles those records cleanly.

Group by the reporting period

```
SELECT
  (happened_at AT TIME ZONE 'Asia/Kolkata')::date AS day,
  COUNT(*) AS events
FROM course.events
GROUP BY (happened_at AT TIME ZONE 'Asia/Kolkata')::date
ORDER BY day;
```

September 1 has two events and September 2 has one. Counts reconcile to three input events. For month-level summaries, DATE_TRUNC('month', local_clock) returns the beginning of the month as a period label. That is a bucket label, not the original event timestamp. Preserve the raw instant when detail-level investigation matters.

Recovery and validation

If a timestamp displays differently on another computer, check the session time zone before assuming the stored instant changed. SHOW TIME ZONE displays the session setting. The explicit conversions above make the displayed comparisons stable regardless of that setting. If a timestamp source lacks an offset or known zone, do not guess one silently; obtain its source convention before treating it as an instant.

Try it yourself

Select the India calendar day September 2 from course.events using explicit timestamptz boundaries. Predict which event sits exactly on its beginning. Save the SQL and explain why that event was excluded from the preceding day's query.

Changed retry - after your first review

Select September 1 according to UTC instead of India. Then explain why a different pair of IDs is correct despite the stored events being unchanged.

Revision in your own words

A calendar date, local clock reading and global instant are different things. State the reporting zone, convert before extracting a local date, and use inclusive-start/exclusive-end windows. Test a record at the beginning, just before the end and exactly at the next beginning. Reconcile grouped counts to the included events.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-05 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-06

Subqueries and nested logic

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:35:16-01:45:58

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-05. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:35:16-01:45:58.](#) [Revisit the](#) subquery visual with this lesson's deeper scalar-cardinality and EXISTS questions. Do not repeat the full video if the earlier watch is clear. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

You have used a scalar average inside a query. Now you will ask a different question: “Which customers have at least one paid order?” The answer should contain each customer once, even if that customer has several paid orders. You will use correlated EXISTS and NOT EXISTS to express that purpose directly. Complete DE03-05.

A correlated subquery refers to the current outer row. In `c.customer_id = o.customer_id`, `c` may belong to the outer query and `o` to the inner query. You can picture the outer customer holding up an ID while the smaller query checks the order stack for a match. This is a reasoning model; it is not a promise that the engine physically rescans the whole order table for each customer.

Follow with me once

```
SELECT c.customer_id, c.customer_name
FROM course.customers AS c
WHERE EXISTS (
    SELECT 1
    FROM course.orders AS o
    WHERE o.customer_id = c.customer_id
    AND o.is_paid = TRUE
)
ORDER BY c.customer_id;
```

C01 Asha and C04 Iqbal qualify. Asha has two paid orders, O101 and O103, but appears once because the outer source has one row per customer and EXISTS returns only a yes/no decision for each. An INNER JOIN to paid orders would produce three matched rows before any further grouping: two Asha rows and one Iqbal row. The choice depends on the requested grain.

The line `o.customer_id = c.customer_id` is essential. If you remove it, the inner query merely asks whether any paid order exists anywhere. That answer is true for this database, so all five customers would pass. Both versions run successfully; only the correlated one checks each customer's own orders.

Find absence without a nullable-list trap

```
SELECT c.customer_id, c.customer_name
FROM course.customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM course.orders AS o
    WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

Only C05 Tara remains. O106's null customer_id does not match Tara or any other known customer. Unlike a nullable NOT IN list, this matching query gives a definite answer about whether a known customer's order exists. Missing outer keys still deserve a declared policy; this outer table has non-null primary keys, so that ambiguity is avoided here.

“Has no orders” and “has no paid orders” are different. The first accepts only Tara. The second would accept Ravi, Mina and Tara: Ravi has an unpaid order, Mina an unknown-payment order, and Tara none. Not having evidence of a paid order does not prove a customer has never paid under a wider historical dataset.

A scalar correlated result

```
SELECT c.customer_id,
    (SELECT COUNT(*) FROM course.orders AS o
     WHERE o.customer_id = c.customer_id) AS order_count
FROM course.customers AS c
ORDER BY c.customer_id;
```

C01 through C05 have counts 2, 1, 1, 1 and 0. COUNT produces one scalar per outer customer. A non-aggregate inner query selecting order_id would return two rows for Asha and fail in that scalar position. Do not choose one arbitrary ID with LIMIT unless the business request specifies which order, with a deterministic ordering rule.

Debugging and recovery

First test the inner logic with one explicit customer ID, then restore the correlation. Check a customer with multiple matches and one with none. If everybody passes EXISTS, inspect for missing correlation. If nobody passes an exclusion using NOT IN, check whether the subquery contains NULL. Understand the output shape before optimising the query.

Try it yourself

Return known customers with no known-paid orders. Keep one row per customer. State why Mina belongs in this result but should not automatically be labelled an unpaid customer.

Changed retry - after your first review

Find customers with at least two orders using a scalar correlated COUNT. Return ID and name, one row per customer. Explain why the count cannot be replaced by EXISTS alone.

Revision in your own words

A correlated subquery uses the current outer record. EXISTS preserves outer grain while testing any match; NOT EXISTS tests no match. Check that correlation is present and that absence means exactly what the business sentence says. A scalar position requires one value, not an arbitrary list of records.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-06 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-07

CTEs and readable multi-step SQL

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:58:55-02:09:07

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-06. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:58:55-02:09:07.](#)

[Watch CTE](#) structure. Use our PostgreSQL query; naming a step does not automatically materialize it. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A correct transformation is easier to review when its steps have names. You will total item lines to order grain, attach those totals to every order, and add each order's shipping fee once. Complete DE03-06. This resolves the fee multiplication measured in DE03-02.

A common table expression, or CTE, is a named query result introduced with WITH for the duration of one statement. A later part of that statement can read the name as a table-like input. It is not automatically a permanent table or a guarantee that PostgreSQL stores an intermediate copy. Picture labelling a tray “item totals by order” so the next calculation can refer to it clearly.

Follow with me once: inspect the first stage

```
WITH item_totals AS (  
  SELECT order_id,  
         SUM(quantity * unit_price) AS item_gross  
  FROM course.order_items  
  GROUP BY order_id  
)  
SELECT * FROM item_totals ORDER BY order_id;
```

There are five rows: O101 129, O102 25, O103 100, O104 250 and O105 110. The CTE's grain is one row per order represented in the item source. Its values total 614. O106 is absent because it has no lines. The name item_totals exists only within this one statement ending at the semicolon.

Follow with me once: preserve order headers

```
WITH item_totals AS (  
  SELECT order_id,  
         SUM(quantity * unit_price) AS item_gross  
  FROM course.order_items  
  GROUP BY order_id  
)  
SELECT o.order_id,  
       COALESCE(i.item_gross, 0) AS item_gross,  
       o.shipping_fee,  
       COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross  
FROM course.orders AS o  
LEFT JOIN item_totals AS i ON o.order_id = i.order_id  
ORDER BY o.order_id;
```

order_id	item_gross	shipping_fee	order_gross
O101	129.00	20.00	149.00
O102	25.00	10.00	35.00
O103	100.00	0.00	100.00
O104	250.00	15.00	265.00
O105	110.00	5.00	115.00
O106	0.00	0.00	0.00

The final grain is one row per order, all six preserved. The missing joined item total means there were no item rows for O106 in this complete classroom fixture. Under the task's rule, an empty order has zero item gross. That is why COALESCE to zero is justified here. In an incomplete delivery, no lines might mean “not received yet”; the same fallback could be misleading. A transformation must know its input completeness rule.

Prove the repaired total

The six order_gross values total 664. The independent source totals are 614 from item amounts and 50 from order fees, also 664. The raw joined calculation from DE03-02 would have produced 689 by multiplying fees. Correcting the final number is not enough: the output must also contain six unique order IDs, including O106.

You can add a second CTE after the first, separated by a comma, then query the final step. Each step should have a grain sentence and a checkable purpose. Avoid putting a semicolon after the first CTE definition because that would end the statement before the consuming SELECT.

Readability does not automatically mean speed

PostgreSQL can inline or materialise CTEs depending on the query and circumstances. A CTE's first benefit here is explainability and a deliberate transformation boundary. DE03-13 introduces EXPLAIN; do not claim that a query is faster just because you put WITH at the front. Avoid repeating the same long calculation across many branches when one named step makes its rule clear.

Try it yourself

Build the same `item_totals` CTE, then return only known-paid orders with `order_id` and `order_gross`, preserving one row per qualifying order. Calculate their combined order gross separately from their item gross and shipping fees.

Changed retry - after your first review

Use `item_totals` to identify order headers with no matching item summary. Return their order IDs and explain why you test the right-side key rather than `item_gross = 0`.

Revision in your own words

WITH names intermediate results within one statement. State each step's grain, aggregate before joining across grains, and preserve required headers with LEFT JOIN. Justify zero fallbacks using the dataset's completeness rule. Validate unique output keys, empty cases and independently scoped totals.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-07 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-08

Window functions and PARTITION BY

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:45:58-01:58:55

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-07. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:45:58-01:58:55.](#) [Watch](#) window functions. Our lesson explicitly teaches partitions, ordered frames and deterministic running totals. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A category total is useful, but sometimes you want to display each sale beside its category total. GROUP BY would collapse the sale detail. You will use a window aggregate to keep each row while adding a related summary. Complete DE03-07.

A window function calculates across a group of related rows while leaving the original result rows available. OVER defines that window. PARTITION BY divides the rows into independent groups for the window calculation. A frame selects which rows within a partition contribute to a particular ordered aggregate. Picture each sale carrying a small card showing its category's total without being merged into a single category receipt.

Follow with me once

```
SELECT order_id, category,  
       quantity * unit_price AS gross_amount,  
       SUM(quantity * unit_price)  
         OVER (PARTITION BY category) AS category_gross  
FROM course.sales  
ORDER BY category, order_id;
```

There are still eight output rows. Paper rows 0007, 0009 and 0011 each display category_gross 184. Their gross_amount values remain 99, 25 and 60. Writing's three rows each show 140; Bags shows 250 and Tools 40. GROUP BY category would have returned four rows instead. Window PARTITION BY establishes a calculation group, not a new output grain by itself.

Do not add the repeated category_gross column down this detailed result to produce a grand total. Paper's 184 appears three times, so that would multiply the category summary. This is the same grain discipline you learned with shipping fees, now applied to an intentional window output.

Calculate a share with a declared denominator

```
SELECT order_id,  
       ROUND(100.0 * quantity * unit_price  
            / NULLIF(SUM(quantity * unit_price)  
                    OVER (PARTITION BY category), 0), 2)  
       AS pct_of_category  
FROM course.sales  
WHERE category = 'Paper'  
ORDER BY order_id;
```

Paper shares are 53.80, 13.59 and 32.61 percent. The 100.0 converts a fraction into a percentage while preserving numeric arithmetic. NULLIF protects a zero denominator. Rounding individual shares can make their displayed sum differ slightly from 100 in other data; validate the unrounded quantities if that occurs.

WHERE is logically applied before window calculations. Filtering to Paper is harmless here because the desired partition is Paper. But if you filter to only order_id = '0007' before the window, the partition contains just that row and its share becomes 100%. To compute the share over all Paper rows and then show one sale, use a CTE containing the window calculation followed by an outer filter.

A running total needs an order and a frame

```
SELECT order_id, ordered_on,  
       quantity * unit_price AS gross_amount,  
       SUM(quantity * unit_price) OVER (  
         ORDER BY ordered_on, order_id  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_gross  
FROM course.sales  
ORDER BY ordered_on, order_id;
```

The running totals are 99, 129, 154, 214, 274, 324, 364 and 614. UNBOUNDED PRECEDING means start at the first row; CURRENT ROW means stop at this row. ROWS uses individual ordered rows. The ID resolves same-date ties. The window ORDER BY controls the calculation sequence, while the final ORDER BY controls display. We write both explicitly.

Without a deliberate frame, the default ordered frame can include peer rows sharing an ordering value, making a running result jump over a whole tied date. Do not memorise a frame as punctuation; explain which rows should contribute to each result.

Try it yourself

For course.practice_sales, display each order ID, category, gross amount and category gross using a window. Keep all four sales. Explain the two Paper rows and why category_gross should not be summed down the detailed output.

Changed retry - after your first review

Produce a running gross total for course.retry_sales by ordered_on, breaking ties with order_id. Use an explicit ROWS frame and predict the total after R202.

Revision in your own words

Window aggregates can preserve detail grain. `PARTITION BY` defines calculation groups; an ordered frame defines contributing rows. `WHERE` affects rows before the window. Use an outer query to filter after computing a full-population window. Repeated partition totals are not additive detail measures.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-08 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-09

ROW_NUMBER, ranking and LAG/LEAD

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:45:58-01:58:55

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-08. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:45:58-01:58:55.](#) [Revisit the](#) ranking portion as needed. The written lesson supplies ROW_NUMBER, tie policy, LAG and LEAD worked examples and outputs. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

You will choose a highest-value sale, handle equal amounts honestly, and compare consecutive records without a self-join. Complete DE03-08. These operations are useful in latest-record selection, deduplication policies and change analysis, but each needs an explicit ordering rule.

ROW_NUMBER assigns a different sequence number to every row in a partition. RANK gives ties the same position and leaves gaps afterwards. DENSE_RANK gives ties the same position without gaps. LAG reads a prior row's value in the specified order; LEAD reads a following row's value. "Previous" means previous row in that order, not automatically previous calendar day.

Follow with me once: ties are a business decision

Use the retry sales because R201 and R203 both have gross amount 75:

```
SELECT order_id, quantity * unit_price AS gross_amount,  
       ROW_NUMBER() OVER (  
         ORDER BY quantity * unit_price DESC, order_id  
       ) AS row_position,  
       RANK() OVER (  
         ORDER BY quantity * unit_price DESC  
       ) AS amount_rank,  
       DENSE_RANK() OVER (  
         ORDER BY quantity * unit_price DESC  
       ) AS dense_amount_rank  
FROM course.retry_sales  
ORDER BY gross_amount DESC, order_id;
```

order_id	gross_amount	row_position	amount_rank	dense_amount_rank
R201	75.00	1	1	1
R203	75.00	2	1	1
R204	30.00	3	3	2
R202	20.00	4	4	3

The ROW_NUMBER includes order_id to choose a deterministic sequence within the amount tie. RANK and DENSE_RANK deliberately rank by amount alone, so the tied 75 values share a rank. If you added the unique ID to their window ordering too, you would break the amount tie and change their meaning. Tie-breaking for presentation and tie-definition for ranking are different choices.

Filter a computed ranking in the next stage

```
WITH ranked AS (
  SELECT order_id, category, quantity * unit_price AS amount,
         ROW_NUMBER() OVER (
           PARTITION BY category
           ORDER BY quantity * unit_price DESC, order_id
         ) AS rn
  FROM course.sales
)
SELECT order_id, category, amount
FROM ranked WHERE rn = 1
ORDER BY category;
```

One highest sale per category appears: Bags 0014/250, Paper 0007/99, Tools 0013/40 and Writing 0010/60. The outer WHERE can use rn because the CTE has already produced it. Trying to filter a window function directly in the same SELECT's WHERE would violate the logical stage order. If the business wants all equal highest sales, use an amount RANK and filter rank 1 instead of choosing one row.

Follow with me once: adjacent records

```
SELECT order_id, ordered_on,
       quantity * unit_price AS amount,
       LAG(quantity * unit_price) OVER (
         ORDER BY ordered_on, order_id
       ) AS previous_amount,
       LEAD(quantity * unit_price) OVER (
         ORDER BY ordered_on, order_id
       ) AS next_amount
FROM course.practice_sales
ORDER BY ordered_on, order_id;
```

P101 has previous NULL and next 80; P102 has previous 18 and next 30; P103 has previous 80 and next 120; P104 has previous 30 and next NULL. P101 and P102 occur on the same date, so the previous record is not the previous day. If you need day-over-day change, aggregate to one row per day first, handle missing days according to the requirement, and then apply LAG.

A missing prior value can mean no prior row or a null value in that prior row. If the distinction matters, also lag the prior row's key. Avoid replacing all such nulls with zero without a defined baseline rule.

Try it yourself

In `course.practice_sales`, select one most recent sale per `customer_id`, ordering by date descending and `order_id` descending within a tie. Keep the missing `customer_id` as a separate unknown bucket for this exercise. Explain why that bucket is not a real identified customer.

Changed retry - after your first review

For `course.retry_sales` in date/ID order, return each amount, its previous amount and the difference from the previous amount. Preserve the first difference as NULL. Explain why filling that first difference with zero would be an added assumption.

Revision in your own words

Use `ROW_NUMBER` to choose one record with a deterministic tie-breaker. Use `RANK` or `DENSE_RANK` when equal measures should share positions. Filter window results in an outer query. `LAG/LEAD` follow row order, not an assumed calendar. Preserve the meaning of missing predecessors and unidentified partition keys.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-09 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-10

Advanced aggregation and multi-grain reasoning

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:43:10-01:52:08

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-09. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:43:10-01:52:08. Optional](#) grouping refresher before the book's full multi-grain worked example. This source does not teach our item-to-order-to-region reconciliation. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

This lesson brings the engineering risk into one complete report. You will return one row per customer region, count orders, add item gross and shipping fees once, retain the unknown-customer bucket and compute an average at the correct grain. Complete DE03-09. A query can use advanced syntax and still multiply money; grain remains the first decision.

Multi-grain reasoning means following measures as records move between different levels of detail. A weighted average uses the appropriate quantity as its weight rather than giving every summary row equal importance. A reconciliation contract names totals and counts expected to agree across equivalent scopes.

Draw the grain changes in words

Item lines start at (order_id, line_no). They first become one item summary per order_id. Order headers already have one row per order_id and supply shipping_fee once. Customers supply region at one row per customer_id. The joined order-level result can finally be grouped to one row per region. Each step has an explicit key; a field's name alone does not tell you whether it is safe to add.

Follow with me once

```
WITH item_totals AS (
  SELECT order_id, SUM(quantity * unit_price) AS item_gross
  FROM course.order_items GROUP BY order_id
), order_totals AS (
  SELECT o.order_id, c.region,
         COALESCE(i.item_gross, 0) AS item_gross,
         o.shipping_fee,
         COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
  FROM course.orders AS o
  LEFT JOIN item_totals AS i ON o.order_id = i.order_id
  LEFT JOIN course.customers AS c ON o.customer_id = c.customer_id
)
SELECT COALESCE(region, 'Unknown') AS region_group,
       COUNT(*) AS order_count,
       SUM(item_gross) AS item_gross,
       SUM(shipping_fee) AS shipping_fees,
       SUM(order_gross) AS order_gross,
       ROUND(AVG(order_gross), 2) AS avg_order_gross
FROM order_totals
GROUP BY region
ORDER BY region_group;
```

Region	Orders	Items	Fees	Order gross	Mean/order
East	3	254.00	30.00	284.00	94.67
North	1	110.00	5.00	115.00	115.00
Unknown	1	0.00	0.00	0.00	0.00
West	1	250.00	15.00	265.00	265.00

The Unknown display label comes from missing customer identity for O106. It is not an actual region supplied by a known customer. Our customers.region is non-null, so that provenance is clear in this fixture. If real data also allowed missing region on a matched customer, you would separate those two reasons when the report needed them.

Prove each level, not just the final total

At order grain, expect six rows and six distinct order IDs. Item totals sum to 614; shipping fees sum to 50; order gross sums to 664. At region grain, four groups must still represent six orders, with the same 614/50/664 totals. East's three orders are O101/149, O102/35 and O103/100. Their sum is 284, which provides a hand-check inside the overall reconciliation.

A wrong join can repeat rows and still accidentally produce a correct total if some amounts are zero. A correct grand total can also hide amounts moved into the wrong region. Therefore check keys, group membership, empty orders, unknowns and amounts together. These are complementary checks, not interchangeable proof.

Why an average of averages can be wrong

The overall average order gross is $664 / 6 = 110.666\dots$, displayed as 110.67. Taking the simple average of the four displayed region averages gives roughly 118.67 instead, because that treats East's three orders like one equally weighted observation and can introduce rounding error too. To combine group means, use each group's count as a weight or, more directly, sum original group totals and divide by sum of counts. Do not average already rounded averages if the original numerators and denominators are available.

The inclusion of O106's zero empty order is part of this task's declared denominator. If the question were average of non-empty orders, the denominator would be five and the mean 132.80. State the population before debating which number is right.

If you are stuck

Temporarily replace the final SELECT with `SELECT * FROM order_totals ORDER BY order_id`. Inspect that intermediate grain. If one order repeats, fix the earlier join or aggregation before touching the final GROUP BY. If a source total and a filtered report disagree, align their scope. If an Unknown group disappears, inspect INNER versus LEFT JOIN and any right-side WHERE predicate.

Try it yourself

Build one row per payment group from `course.orders`, preserving Paid, Unpaid and Unknown. Use item totals at order grain before adding fees. Return `order_count` and `order_gross` for each group, then reconcile to all six orders and 664 total gross.

Changed retry - after your first review

Using the same item-to-order preparation, report the count and average gross for non-empty orders only. Independently calculate total gross divided by that order count. Explain the difference from the all-order average in the guided report.

Revision in your own words

Track grain and key at every stage. Aggregate line measures before combining header measures. Preserve explicitly required empty and unknown cases. Reconcile counts and measures at equal scopes. Combine group totals and counts for a weighted mean; do not blindly average averages.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-10 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-11

PostgreSQL setup, schemas and dialect awareness

Watch first

[1. SQL for Data Analytics - Learn SQL in 4 Hours](#)

01:26:18-01:32:48

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-10. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: SQL for Data Analytics - Learn SQL in 4 Hours, 01:26:18-01:32:48.](#)
[Observe](#) PostgreSQL setup if you need orientation. Your database is already prepared before M02: do not reinstall. Schemas, search_path and dialect checks are taught in our book. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this lesson comes here

The SQL Desk Setup companion deliberately provided the minimum installation before M02. You have already used a working server rather than waiting until this lesson to execute your first query. Now you will deepen the database model, inspect object names and types, and understand why an instructor's command can need translation. Complete DE03-10. This canonical lesson retains its place in the original curriculum; it is not a request to reinstall working software.

A connection identifies a server, port, database and login role. A schema is a namespace inside one database. A qualified name includes the schema, such as course.orders. A dialect is the particular SQL syntax and behaviour implemented by a database product. DDL, data definition language, defines objects; DML, data manipulation language, changes stored records. SELECT reads records.

Follow with me once: know where the query runs

```
SELECT current_database() AS database_name,  
       current_user AS role_name,  
       current_schema() AS default_schema;  
SHOW server_version;  
SHOW search_path;
```

On your laptop database_name should be standalone_de, while role_name is the role used for your connection. The default_schema and search_path depend on configuration. Common installations search a user-named schema if it exists, then public. Do not assume course is automatically first. The authoring test engine's own database name can differ; that is not the value you are asked to reproduce on your installed server.

search_path determines which schemas PostgreSQL searches for an unqualified object name. If two schemas each contain orders, FROM orders may select a different table when the search path changes. FROM course.orders makes the intended schema explicit. A schema prefix does not cross into another database; you choose a different database through the connection.

Inspect types without guessing from formatting

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'course' AND table_name = 'orders'
ORDER BY ordinal_position;
```

information_schema exposes standard metadata about database objects. Metadata means information describing data: column names, types and nullability in this case. Expect order_id text/non-null, customer_id text/nullable, ordered_on date/non-null, shipping_fee numeric/non-null and is_paid boolean/nullable. The returned type name numeric does not itself show the chosen precision and scale; those have their own metadata fields.

IDs are text because arithmetic does not describe their role and leading zeros may matter. quantity uses integer because fractional notebooks are not valid in this fixture. unit_price uses numeric(10,2): a decimal type with ten total digits and two fractional digits. A type restricts representation, but a sensible range may also need a constraint. An integer quantity could be negative without a separate rule.

Make one disposable table and explain every part

A temporary table belongs to your database session and normally disappears when that session ends. It is useful for a small isolated experiment. Use this unique name in a fresh Query Tool session:

```
CREATE TEMP TABLE desk_note (
  note_id text PRIMARY KEY,
  note_text text NOT NULL
);
INSERT INTO desk_note (note_id, note_text)
VALUES ('N01', 'A SQL file is saved text.');
```

CREATE defines the columns. PRIMARY KEY declares a non-null unique identity. NOT NULL requires a value. INSERT supplies one record, naming the target columns to make their order clear. SELECT reads it. The result is N01 with the sentence shown. These actions differ: saving the script creates a file, executing CREATE creates a database object, and executing INSERT stores a record in that object.

If you rerun CREATE in the same session, “already exists” is expected. Inspect the existing temporary table or use a new temporary name. Do not drop course.orders to fix an unrelated temporary-table experiment. Keeping setup, transformation and evidence files separately named helps you avoid executing a destructive setup command by accident.

Translate the instructor's syntax deliberately

Instructor's MySQL pattern

PostgreSQL form used here Meaning to preserve

USE database_name Connect Query Tool to that database

Choose the database

Backticks around names Simple lowercase names, or double quotes when needed

Identify an object or column

MONTH(date_column) EXTRACT(MONTH FROM date_column)

Calendar month number

IFNULL(value, fallback) COALESCE(value, fallback) Declared missing-value fallback

HAVING output_alias > 10 HAVING aggregate_expression > 10

Filter grouped results

LIMIT offset, count LIMIT count OFFSET offset Skip and cap ordered rows

Single quotes remain the form for text literals. Double quotes preserve an identifier's exact spelling and case in PostgreSQL; creating a mixed-case quoted name means you must reference that exact name deliberately. Our lowercase names avoid that extra beginner burden. Never assume a syntax translation changes only spelling: integer division, null ordering, date operations and window defaults can also need inspection.

Try it yourself

Inspect information_schema.columns for course.order_items. Record its columns, types and nullability. Explain why order_id alone is not the item-line key and why both key fields are required together.

Changed retry - after your first review

Create a temporary table called desk_retry with text reference as primary key and a non-null integer units column. Insert X01 with units 3, then read it. Explain whether merely saving those statements to a file creates the table.

Revision in your own words

Know your connection's server, database and role. A schema is a namespace, and qualification avoids search-path ambiguity. Metadata reveals types and nullability. CREATE defines an object, INSERT adds records, SELECT reads them. Translate dialects by meaning and verify the result; do not install a new database for every video.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-11 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-12

Constraints and transactions

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:29:24-02:36:26

[2. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:40:55-02:54:45

[3. PostgreSQL Demonstration: Transactions](#)

00:00-19:35

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-11. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:29:24-02:36:26. Watch](#) primary keys; the book explains our exact contracts. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:40:55-02:54:45. Watch](#) UNIQUE and CHECK constraints. Read the book's important distinction between CHECK and NOT NULL. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: PostgreSQL Demonstration: Transactions, 00:00-19:35. Observe a real PostgreSQL](#)

transaction demonstration. The instructor uses a terminal and additional database operations; reproduce only our temporary-table transaction lab. Our book explains BEGIN, COMMIT, ROLLBACK and recovery from an aborted transaction. Stop manually at the listed end. Public instructor identity, topic description and player runtime inspected. Full video assigned because no publisher chapter boundaries were provided; full playback not verified.

Why this matters, and what you will do

A reliable database should reject impossible records, and a multi-step change should not stop halfway with a misleading state. You will enforce a simple data contract and practise both committing and rolling back changes in temporary tables. Complete DE03-11. This lesson uses disposable objects; it does not ask you to alter the master sales source.

A constraint is a rule enforced by the database. PRIMARY KEY enforces a unique non-null identity. FOREIGN KEY requires a non-null reference to match an allowed key in another table. CHECK tests a condition, and NOT NULL explicitly forbids missing values. A transaction groups statements into a unit: COMMIT accepts their changes, while ROLLBACK cancels uncommitted changes in that transaction.

Follow with me once: write the contract into a table

```
CREATE TEMP TABLE stock_contract (  
  product_id text PRIMARY KEY,  
  units integer NOT NULL CHECK (units >= 0)  
);  
INSERT INTO stock_contract VALUES ('PEN', 10), ('BAG', 4);  
SELECT * FROM stock_contract ORDER BY product_id;
```

BAG/4 and PEN/10 should appear. The constraints encode distinct rules: each product ID identifies one record, units must be supplied, and units cannot be negative. CHECK alone is not a replacement for NOT NULL because PostgreSQL accepts a check condition that evaluates to unknown; the explicit NOT NULL closes that gap.

Run this deliberately invalid statement by itself. It should fail with a check-constraint violation; no NEG record should be inserted.

```
INSERT INTO stock_contract VALUES ('NEG', -1);
```

The number 23514 is PostgreSQL's SQLSTATE for this error class, not part of the query. A duplicate product ID should trigger a unique violation; a null units value should trigger a not-null violation. Constraints are useful because they reject the bad write rather than hoping every later report notices it.

Follow with me once: rehearse a change and cancel it

```
BEGIN;  
UPDATE stock_contract SET units = units - 2  
WHERE product_id = 'PEN';  
SELECT * FROM stock_contract WHERE product_id = 'PEN';  
ROLLBACK;  
SELECT * FROM stock_contract WHERE product_id = 'PEN';
```

Inside the transaction you see PEN/8. After ROLLBACK, you see PEN/10 again. UPDATE changes existing records selected by WHERE. Without WHERE, every record would be updated, which would be a different operation. Previewing the intended record with SELECT before modifying it is a sensible habit.

ROLLBACK cancels changes still uncommitted in the current transaction. It is not an undo button for changes committed yesterday. If pgAdmin auto-commits a standalone UPDATE, that change is already committed unless you began an explicit transaction. This is why the lesson writes BEGIN and ROLLBACK visibly.

Atomicity: the whole unit succeeds or is cancelled

Imagine transferring two units between two stock records. One decrease and one increase belong together. Atomicity means that the unit is accepted together or rolled back; the intermediate half-transfer must not be mistaken for a final state. Consistency here means declared constraints still hold. Isolation concerns what concurrent transactions can observe, and durability concerns accepted changes surviving normal failures. Those four terms form ACID. This small one-session exercise demonstrates commit/rollback and constraint enforcement, not all concurrent-isolation behaviour.

Try an explicit transaction containing a valid update and then the invalid negative insert. After the failed statement, PostgreSQL marks that transaction aborted. Later ordinary statements fail

until ROLLBACK. Run ROLLBACK alone to recover; the earlier uncommitted update is cancelled too. Do not keep executing unrelated queries in the aborted transaction and treat each resulting error as a new independent problem.

Foreign keys preserve relationships, not every business rule

course.order_items.order_id references course.orders.order_id. A non-null item order ID must correspond to an order header. Nullable customer_id on orders still allows an unidentified customer; a foreign key does not automatically require a value. Nor does it prove that a recorded payment is true, a product description is accurate, or an imported source is complete. Constraints enforce the rules you actually declared.

Try it yourself

Create a temporary table payment_contract with payment_id text primary key and amount numeric(10,2) non-null, checked to be at least zero. Insert PAY1/50. Start a transaction, update PAY1 to 60, inspect it, and roll back. Then confirm the stored amount. Explain why the final value differs from the within-transaction value.

Changed retry - after your first review

In a new temporary stock table, insert ONE/5. Begin a transaction, reduce units to 4 and COMMIT. Verify 4 remains. Attempt a second insertion with the same key and record the error without removing the original record. Explain why ROLLBACK after the successful commit cannot restore the earlier 5.

Revision in your own words

Use separate constraints for identity, missingness, range and relationships. CHECK does not automatically forbid NULL. BEGIN opens a transaction, COMMIT accepts it and ROLLBACK cancels its uncommitted work. Recover an aborted transaction with ROLLBACK and then fix the first error. Constraints do not replace source-quality checks or a clear business contract.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-12 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-13

Indexes and EXPLAIN basics

Watch first

[1. Lesson 3, PostgreSQL Indexes and B-Trees \(and an introduction to EXPLAIN\)](#)

10:10-18:00

[2. Lesson 3, PostgreSQL Indexes and B-Trees \(and an introduction to EXPLAIN\)](#)

29:38-34:18

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-12. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Lesson 3, PostgreSQL Indexes and B-Trees \(and an introduction to EXPLAIN\)](#), 10:10-18:00. Watch index purpose and the EXPLAIN introduction. Our reproducible lab uses a small 10,000-row temporary table; plan choices vary by data and environment. Stop manually at the listed end. Public title, creator, runtime and published chapter boundaries inspected; full playback not verified.

[Instructor demonstration: Lesson 3, PostgreSQL Indexes and B-Trees \(and an introduction to EXPLAIN\)](#), 29:38-34:18. Picture the B-tree structure and search. Do not install TimescaleDB for this course exercise. Stop manually at the listed end. Public title, creator, runtime and published chapter boundaries inspected; full playback not verified.

Why this matters, and what you will do

A query that works on eight records may perform poorly on a much larger table. You will inspect a query plan, create one targeted index in a temporary experiment and compare what PostgreSQL chooses. Complete DE03-12. Correct results come first; optimisation must preserve the same question and records.

An index is an extra data structure that can help locate rows without scanning every table row. A query plan describes the operations the engine intends to use. A sequential scan inspects the table; an index scan uses an index path. Selectivity describes how narrowly a condition filters the source. Statistics help the planner estimate result sizes and costs.

Picture a book's index: looking up one rare topic may be quicker through page references. Reading almost the entire book may be easier in page order. Maintaining the index also takes space and work when the book changes. Database indexes likewise are not a free universal speed button.

Follow with me once: build an isolated experiment

```
CREATE TEMP TABLE plan_orders AS
SELECT n AS order_no,
       n % 100 AS customer_no,
       n * 1.0 AS amount
FROM generate_series(1, 10000) AS g(n);
ANALYZE plan_orders;
EXPLAIN SELECT order_no, amount
FROM plan_orders WHERE order_no = 8765;
```

CREATE TABLE AS stores a query's result as a table. generate_series supplies integers 1 through 10000; g(n) names the generated source and its column. The % operator gives the remainder, producing repeated customer numbers for a later comparison. ANALYZE collects table statistics. It is unrelated to a business analysis report despite sharing the word.

Before an index exists on order_no, expect a sequential scan with a filter. The exact text, cost numbers and timings depend on engine version, configuration and machine. The count estimate should be inspected as an estimate, not copied into evidence as an actual measured row count.

```
CREATE INDEX plan_orders_order_no_idx ON plan_orders(order_no);
ANALYZE plan_orders;

EXPLAIN (ANALYZE, BUFFERS)
SELECT order_no, amount
FROM plan_orders WHERE order_no = 8765;
```

The planner may now select an index path for this narrow equality query. EXPLAIN ANALYZE actually executes the SELECT and adds measured observations. BUFFERS reports buffer activity. The result query itself returns order_no 8765 and amount 8765; EXPLAIN displays plan information instead of that ordinary data grid. Run the SELECT separately to verify correctness.

Read only the plan facts you understand

Look for the scan type, filter or index condition, estimated rows versus actual rows, and the actual number of loops. Cost units are planner units, not milliseconds. Actual time is measured for this execution, not a guaranteed production latency. Cache state and timing overhead can affect a small benchmark. A native Windows server may choose or time the plan differently from the author's PostgreSQL WASM test engine.

EXPLAIN without ANALYZE plans a statement without executing its data operation. EXPLAIN ANALYZE does execute it. Do not casually apply ANALYZE to an UPDATE or DELETE that you did not intend to run. Here we use SELECT against a temporary fixture only.

Compare a broad request

```
EXPLAIN SELECT order_no, amount
FROM plan_orders WHERE order_no >= 1;
```

Every row matches, so a sequential scan may be the efficient choice even though an index exists. Its presence does not force use. Indexes also need storage and maintenance during inserts, updates and deletes. Choose them for an observed workload, then validate both the result and the plan. An eight-row teaching table usually gives too little evidence for a serious performance claim.

If you are stuck

An “already exists” error can mean you reran the temporary setup in the same session. Use a fresh Query Tool session for a clean experiment rather than modifying the course's permanent tables. If an index does not appear in a plan, inspect selectivity, statistics and what the query actually asks. Do not disable sequential scans just to obtain the plan shape you hoped to see.

Try it yourself

Inspect the query for `order_no = 4321` after the guided index exists. Save its `SELECT` result and `EXPLAIN (ANALYZE, BUFFERS)` output. Explain one actual-versus-estimated observation without claiming that a single run proves production speed.

Changed retry - after your first review

Inspect a query selecting `customer_no = 21`, for which no `customer_no` index has been created. Predict the number of matching rows, execute a `COUNT` to check it, and inspect the plan. Explain why an index on `order_no` does not automatically become an index on `customer_no`.

Revision in your own words

Indexes offer access paths with storage and write costs. `EXPLAIN` shows a plan; `EXPLAIN ANALYZE` executes and measures. Compare estimated and actual rows, but do not read planner cost as milliseconds. Check a narrow and a broad query, preserve correctness, and report what your environment actually did.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-13 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-14

Views and reusable database logic

Watch first

[1. PostgreSQL Tutorial Full Course 2022](#)

01:34:22-01:45:01

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-13. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: PostgreSQL Tutorial Full Course 2022, 01:34:22-01:45:01. Watch ordinary](#) PostgreSQL views. Use the book's corrected view examples; stored functions beginning at the stop time belong to later study. Stop manually at the listed end. Public title, creator, runtime and published chapter boundaries inspected; full playback not verified.

Why this matters, and what you will do

You now have a useful order-total calculation. Several reports might need it. You will give that calculation a reusable database name, understand when its results are computed, and distinguish a view from a stored snapshot. Complete DE03-13.

A view stores a query definition and exposes its result through a table-like name. An ordinary view does not store a separate durable copy of the result rows. A materialized view stores results and needs a deliberate refresh to bring them up to date. A CTE names a result only within one statement; a view remains as a database object until removed, subject to its scope and permissions. We create temporary views here so the experiment stays within the current session.

Follow with me once

```
CREATE TEMP VIEW order_totals_view AS
WITH item_totals AS (
  SELECT order_id, SUM(quantity * unit_price) AS item_gross
  FROM course.order_items GROUP BY order_id
)
SELECT o.order_id, o.customer_id, o.is_paid,
       COALESCE(i.item_gross, 0) AS item_gross,
       o.shipping_fee,
       COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
FROM course.orders AS o
LEFT JOIN item_totals AS i ON o.order_id = i.order_id;

SELECT order_id, order_gross
FROM order_totals_view ORDER BY order_id;
```

The six totals remain O101 149, O102 35, O103 100, O104 265, O105 115 and O106 0. You can now query `order_totals_view` without retyping its entire definition. Its contract is still one row per order with the same complete-fixture rule for empty orders. The convenient name does not remove the need to validate its logic.

```
SELECT COUNT(*) AS orders,  
       COUNT(DISTINCT order_id) AS distinct_orders,  
       SUM(order_gross) AS gross  
FROM order_totals_view;
```

Expect 6, 6 and 664. These checks belong to the view's contract and can be reused after a change. A view is not automatically a security boundary or performance improvement. Permissions and execution details need their own design, taught more deeply later.

Show that an ordinary view reads current underlying data

Use a separate tiny temporary table so the main training records remain intact:

```
CREATE TEMP TABLE view_source (code text, units integer);  
INSERT INTO view_source VALUES ('A', 2);  
CREATE TEMP VIEW view_demo AS  
SELECT code, units, units * 10 AS value FROM view_source;  
SELECT * FROM view_demo;  
UPDATE view_source SET units = 3 WHERE code = 'A';  
SELECT * FROM view_demo;
```

Before the update, the view returns A/2/20; after it, A/3/30. You did not refresh an ordinary view or edit its stored result because it represents a query over the current source. A materialized view would retain its stored result until refreshed; a historical CSV export would also remain an old snapshot until regenerated.

A stable contract is more than a convenient name

Downstream queries can depend on the view's column names, types, meanings and grain. Changing `order_gross` from fee-inclusive to item-only without changing its contract would silently mislead those readers. `CREATE OR REPLACE VIEW` can update a definition within PostgreSQL's permitted compatibility rules, but replacing syntax does not substitute for reviewing the meaning change. Keep a clear definition and validate existing consumers when a shared view changes.

A view definition should not be relied on to force final display order. The query reading it should use its own `ORDER BY`. Likewise, selecting a subset of its columns does not remove the underlying rule that determines its rows.

If you are stuck

If a temporary view disappears in another Query Tool tab, remember that it belongs to its creating session. Run its creation script in the session where you want to use it. If you need a persistent shared view later, create it in a deliberately chosen schema with appropriate permissions; do not assume the temporary tutorial object was permanent. If rows duplicate, investigate the definition rather than adding `DISTINCT` at every consumer.

Try it yourself

Query `order_totals_view` for known-unpaid orders, retaining empty orders. Return ID and gross, then count those orders and sum their gross. State why a count of one would be wrong even if the amount still looked right.

Changed retry - after your first review

Using the disposable `view_source` table and `view_demo`, begin a transaction, change units from 3 to 4, inspect the view, then roll back and inspect it again. Explain why the view follows the transaction's visible source state.

Revision in your own words

An ordinary view stores a query definition, a materialized view stores results, and a CTE is scoped to one statement. Temporary views are session-scoped. Publish clear grain, meaning, field and null-handling contracts; test counts and totals. A reusable name does not guarantee correctness, security, ordering or speed.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-14 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE03-15

Integrated unseen SQL challenge

Watch first

[1. Learn PostgreSQL - complete beginner course](#)

Optional visual - no verified bounded range. Use this as a SQL and database workflow refresher. COPY and backup/restore commands follow the current PostgreSQL instructions printed in the lesson. For a cold assessment, revisit after your first attempt.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-14. Video: Independent challenge; revisit earlier assigned videos only after an attempt. Read the vocabulary, perform the guided example and save an independent attempt before review.

What this lesson measures

You have learned the component skills. Now you will combine them on a new fictional delivery with different amounts, repeated fees and an empty order. Complete DE03-14. This is a practical challenge, so do not watch a solution video first. If a skill needs refreshing, revisit its earlier assigned segment, record that help and then begin a fresh attempt. There is no new syntax hidden in the task.

Run labs/SQL/01_load_assessments.sql once on your standalone_de database after the initial load. It creates assessment inputs without solutions. If the tables already exist, inspect them rather than rerunning the loader; use the setup guide's recovery instructions if a failed load left a transaction open. The course reset removes these assessment tables too, so after a deliberate reset both load files need to be rerun in order.

Understand the new delivery

Use course.challenge_customers, course.challenge_orders and course.challenge_items. Customer grain is customer_id, order grain is order_id, and item grain is (order_id, line_no). There is no item name because this task needs quantity and price, not product classification. A missing customer_id stays unknown. An order with no item rows is an intentionally empty order in this complete fixture, so its item amount is zero while its actual fee still counts.

Table	Fields to inspect before coding
challenge_customers	customer_id, region
challenge_orders	order_id, customer_id, ordered_on, shipping_fee, is_paid
challenge_items	order_id, line_no, quantity, unit_price

Read the inputs with SELECT statements and save their counts. This is permitted preparation; the challenge is constructing and validating the transformation rather than guessing unseen source values. Do not open the matching Explained Reviews section until your attempt is saved.

A method you can follow without an answer

First write one sentence for the desired final grain. Name each amount's source grain before deciding where it can be added. Build and inspect the item summary alone, then the order-level result, and only then the final summary. Keep an empty-order case visible. Finally, check independent totals from the original tables so a mistake in the same prepared result does not validate itself.

A helpful help-request format is: "DE03-15, I expect one row per order; my intermediate query returns ..., and I think the mismatch begins at ...". Include the SQL and actual output. This gives your tutor something concrete to diagnose and makes the amount of assistance visible.

Try it yourself

Save DE03-15_attempt.sql with numbered queries and a short evidence file.

1. Build one row per order with item_gross, shipping_fee, order_gross, customer_id and all three payment states preserved. Keep every order, including the empty one.
2. Return one row per region, displaying Unknown for missing customer identity. Include order_count and order_gross. Reconcile the count, item amount, fees and final amount to their original sources.
3. Return every order tied for highest order_gross within its region. Preserve ties by amount rather than arbitrarily choosing one. Explain the difference from ROW_NUMBER.
4. Identify customers with no orders. Explain why this is different from customers with no paid orders.
5. Explain why SUM(DISTINCT shipping_fee) is unsafe on this delivery. Use the actual repeated fee values as evidence.

Before review, record what you could solve independently, any help used and the remaining uncertainty. A query that executes but loses an order or misclassifies a payment is not ready merely because the totals look plausible.

Changed retry - after your first review

Use course.challenge_retry_customers, course.challenge_retry_orders and course.challenge_retry_items. Build all-order totals and region totals from a blank editor. Keep empty orders and unknown identity, reconcile counts and amounts, and calculate the average gross per order over the complete order population. Explain why averaging region averages equally would be unsafe. Do not reuse the previous result numbers.

Revision in your own words

For an unfamiliar source, inspect its fields and grain before writing the final report. Construct a validated order-level result, then summarise or rank it. Empty orders may still have fees. Repeated equal fees are legitimate on different orders. Save evidence of independence and repair, not just a polished final query.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-15 in the Explained Reviews book. After review, solve the changed retry

and keep it as a separate attempt.

DE03-16

Timed SQL interview set and explanation

Watch first

[1. Learn PostgreSQL - complete beginner course](#)

Optional visual - no verified bounded range. Use this as a SQL and database workflow refresher. COPY and backup/restore commands follow the current PostgreSQL instructions printed in the lesson. For a cold assessment, revisit after your first attempt.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE03-15. Video: Independent challenge; revisit earlier assigned videos only after an attempt. Read the vocabulary, perform the guided example and save an independent attempt before review.

What this lesson measures

An interview adds communication and limited time to the SQL skills you have practised. You will explain assumptions, solve a small set of questions and review accuracy before speed. Complete DE03-15 and its changed retry. A stopwatch is an optional measurement tool for this rehearsal, not a forced study timer or an automatic mastery switch.

First attempt the set untimed if time pressure makes it hard to think. When you can solve it independently, repeat a changed set with a 45-minute total budget: five minutes to inspect the task, thirty to write and run queries, and ten to validate and explain. Those times are a practice arrangement, not a universal employer format. Record actual time and help used; finishing quickly does not compensate for wrong grain or lost unknowns.

How to talk while solving

Start with the question in plain language: “One output row per customer; dates choose recency; ID breaks same-date ties.” State any ambiguity before committing to a query. Explain a small plan, then type. If an error appears, identify the stage or missing assumption instead of apologising repeatedly. A useful explanation tells the interviewer why the result is correct, not only which SQL keywords you remember.

You may consult table definitions and your own prior work during the untimed learning attempt. For the timed rehearsal, use only the supplied source definitions, your query editor and its error messages. If you open an answer, label that portion assisted and repair it later on the changed set. This book does not award an external interview pass from a self-reported timer.

Try it yourself

Use the original course.sales, course.customers, course.orders and course.order_items tables. Save one .sql file with four numbered answers and a brief spoken-or-written explanation for each.

1. From `course.sales`, return the latest sale per known customer, with date descending and `order_id` descending as the tie-breaker. Exclude missing customer identity explicitly. Keep one record per known customer.
2. From `course.sales`, show categories with at least two sales and gross total above 150. State the input and output grain and why the conditions use `HAVING`.
3. From `orders` and `order_items`, return all orders with `item_gross` and fee-inclusive `order_gross`, preserving empty orders. Supply an independent amount reconciliation and a unique-order check.
4. Diagnose this claim: “`COUNT(is_paid)` counts paid orders, and adding `DISTINCT` after a join always prevents duplicate money.” Give one concrete counterexample for each using the course data.

At the end, choose the least certain query and explain a specific changed input that would test it. Do not quietly repair your saved first attempt after reading the review; save the repair separately.

Changed retry - after your first review

Use `course.practice_sales` for two fresh interview questions: choose the highest-gross sale per category, resolving ties by ascending ID; then return categories whose known-paid gross exceeds 50. Explain when `WHERE` happens relative to `GROUP BY` and a window calculation. For a third task, use `course.challenge_retry` tables to prove the all-order total and state its average denominator. Save the time and assistance record separately from the first set.

Revision in your own words

Interview practice measures correct reasoning under an agreed constraint, not typing speed alone. State grain, null policy, tie rules and scope, then verify actual outputs. Keep a timed attempt honest about assistance. After this lesson and its repairs, attempt G02; M03 completion alone does not automatically pass the gate.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE03-16 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.